Lane Detection

Massimo Violante
Politecnico di Torino
Dip. Automatica e Informatica

Outline

- Introduction
- The image pipeline
  - Camera calibration
  - Distortion correction
  - Perspective transformation
  - Lane detection

Outline

- Introduction
- The image pipeline
  - Camera calibration
  - Distortion correction
  - Perspective transformation
  - Lane detection

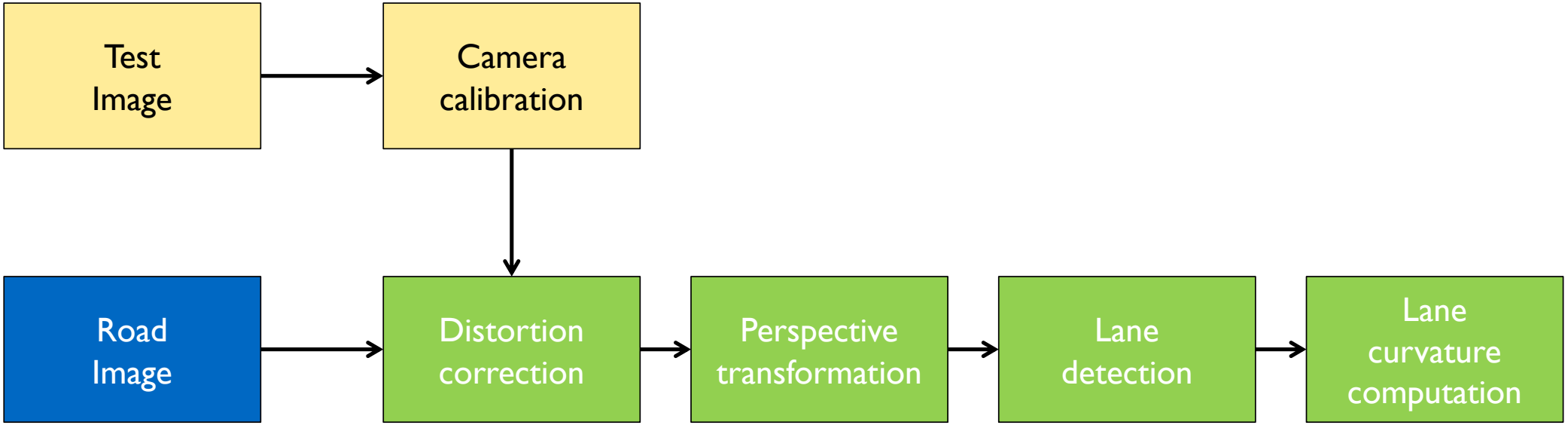
Introduction

- Lane detection is crucial for driver assistance systems and for autonomous vehicles
- It is used to compute the position of the vehicle w.r.t. to the lane lines
- The information can be used:
  - To warn the driver in case of potential involuntary lane crossing
  - To control the steering to maintain a desired position w.r.t. the lane lines
  - To compute the radius of curvature of the lane

Outline

- Introduction
- The image pipeline
  - Camera calibration
  - Distortion correction
  - Perspective transformation
  - Lane detection

The image pipeline



Outline

- Introduction
- The image pipeline
  - Camera calibration
  - Distortion correction
  - Perspective transformation
  - Lane detection

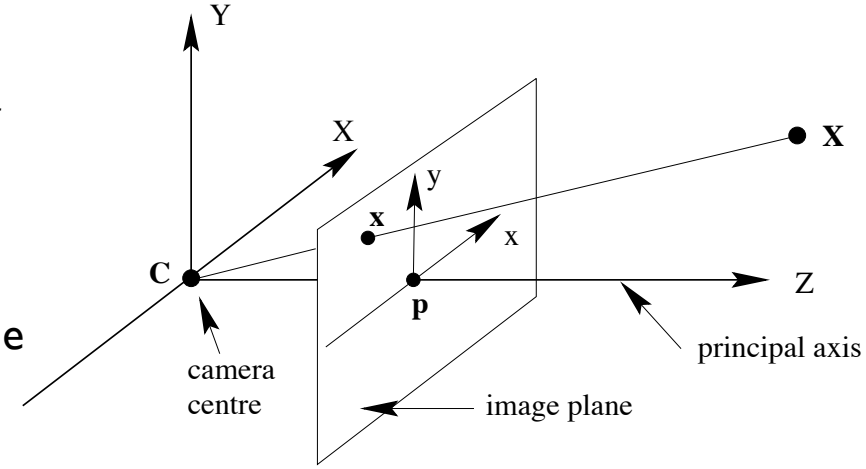
Camera calibration

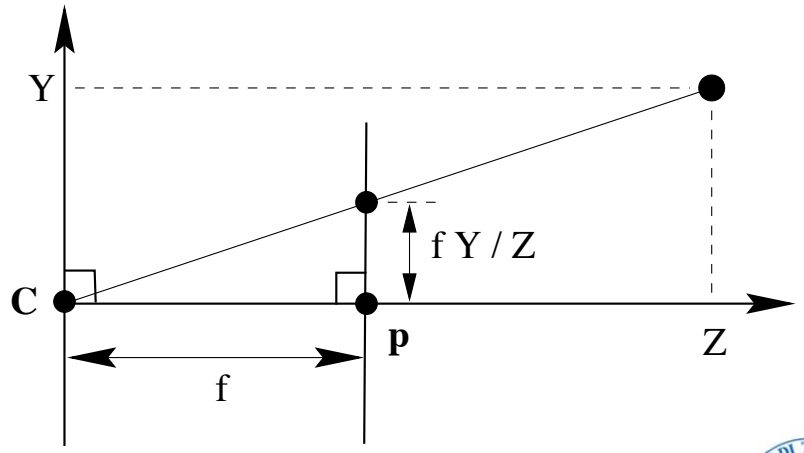
- A camera is a mapping between the 3D world (object space) and a 2D image
- Camera can be modelled using matrices with properties that represent the camera 3D to 2D mapping
- The basic model is the pinhole model

Pinhole model

- We consider the central projection of points in space onto a plane
  - Centre of projection = origin of a Euclidean coordinate system
  - Plane Z = f is called the image plane or focal plane
- A point in space $\mathbf{X} = (X, Y, Z)^T$ is mapped to the point on the image plane where a line joining the point $\mathbf{X}$ to the centre $\mathbf{C}$ of projection meets the image plane
- The point $(X,Y,Z)^T$ is mapped to the point $(fX/Z, fY/Z, f)^T$ on the image plane
- The mapping from Euclidean 3-space to Euclidean 2-space is (by ignoring the third image coordinate)

$$(X, Y, Z)^T \mapsto (fX/Z, fY/Z)^T$$





Homogeneous coordinates

- Homogeneous coordinates are a projective geometry representation that extends ordinary (Euclidean) coordinates so that:
  - Translations become linear operations
  - Points at infinity can be represented
  - Perspective projections can be expressed as matrix multiplication

- Example
  - Euclidean point: (x_e, y_e)
  - Corresponding homogeneous coordinates: (x, y, w)
  - Conversions: $x_e = \frac{x}{w}, y_e = \frac{y}{w}$
  - Usually $w = 1$, so we have: $(x, y) \rightarrow (x, y, 1)$

Translations and Rotations

- Translation
  - We want to express the translation of the point (x, y) to the point $\left(x^t = x + t_x, y^t = y + t_y\right)$
  - Using homogenous coordinates, we can express the operation as a Matrix multiplication

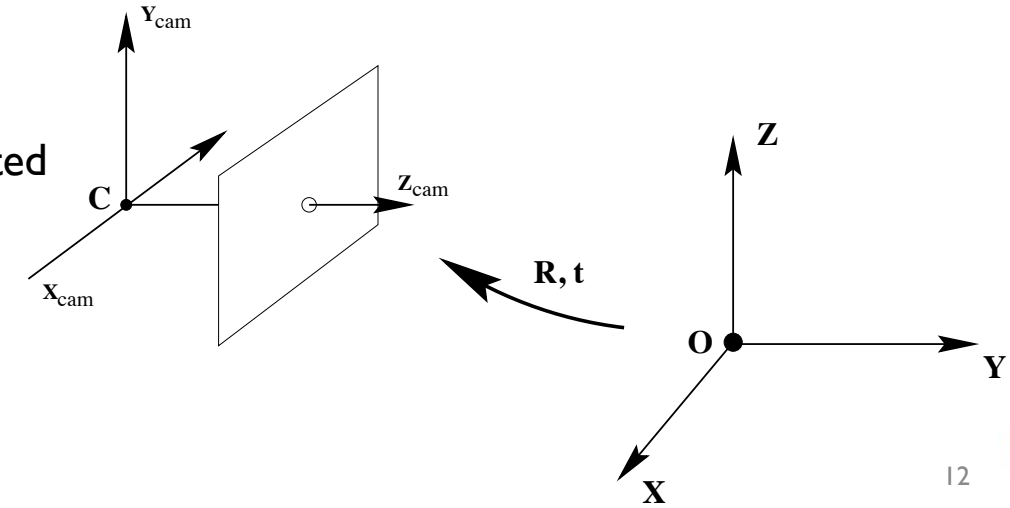
$$\begin{bmatrix} x^t \\ y^t \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & & t_x \\ & 1 & t_y \\ & & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Rotation
  - We want to express the rotation using rotation matrix $R$of the point (x, y, z)

$$\begin{bmatrix} x^R \\ y^R \\ z^R \\ 1 \end{bmatrix} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Pinhole camera model

- A model of a camera can be built starting from its characteristics
- Assumptions:
  - The camera center is defined as C
  - The image plane is located at distance f (focal length) from the pinhole
  - We have a camera coordinate system where:
    - The origin is the camera center
    - z axis point forward
    - The image plane is perpendicular to the z axis
  - The 3D world has its origin in O
  - The projection of the 3D world point is rotated using rotation matrix R and translated by t



Pinhole camera model

- 3D world point in the 3D world reference system $\tilde{X}_w = \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$

- 3D world point in the camera reference system $\tilde{X}_c = \begin{bmatrix} X_C \\ Y_C \\ Z_C \\ 1 \end{bmatrix}$

- The image point $\tilde{x} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

Pinhole camera model

- The transformation from 3D world coordinates to camera coordinates is $X_c = RX_w + t$ that in homogenous coordinates becomes

$$\tilde{X}_C = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \tilde{X}_W$$

- The image plane is defined by:
  - Focal length in pixels: (f_x, f_y)
  - Principal point (center of the sensor) in pixels: (c_x, c_y)
- The intrinsic calibration matrix conveys the image characteristics

$$K = \begin{bmatrix} f_x & & c_x \\ & f_y & c_y \\ & & 1 \end{bmatrix}$$

Pinhole camera model

- The projection of the 3D point in the camera reference system on the image plane becomes:

$$\tilde{x} = K \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

- Putting all together, we have:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & & c_x \\ & f_y & c_y \\ & & 1 \end{bmatrix} \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

Pinhole model

- The equation is normally written as

$$\tilde{x} = K[R\ t]\tilde{X}_w$$

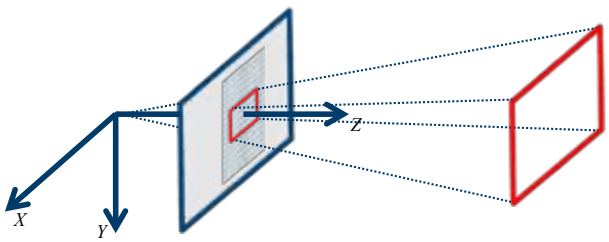
- $P = K[R\ t]$ is the pinhole camera model

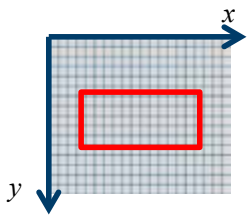
- K is the intrinsic (internal) camera calibration matrix, that defines the properties of the camera

- $[R\ t]$ is the extrinsic (external) calibration matrix, that defines the location of the camera in the vehicle

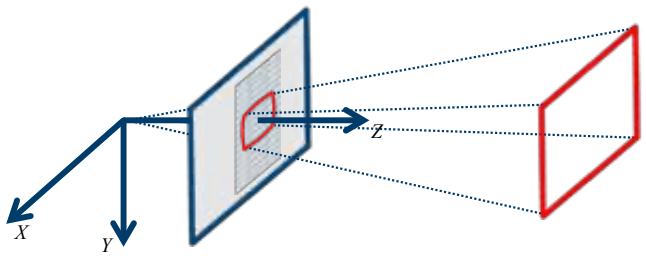
Camera distortion

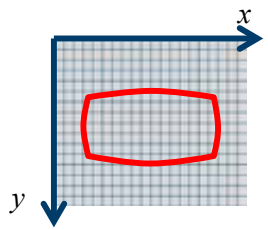
- The pinhole model is ideal, as it does not consider distortion





No radial distortion



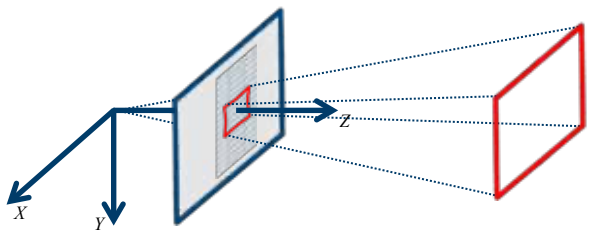


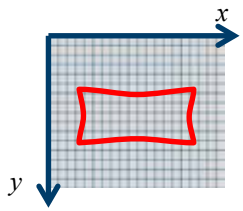
Barrel distortion

- Radial distortion correction

$$\hat{x} = x(1 + \kappa_1(x^2 + y^2) + \kappa_2(x^2 + y^2)^2)$$
$$\hat{y} = y(1 + \kappa_1(x^2 + y^2) + \kappa_2(x^2 + y^2)^2)$$

- k_1 and k_2 are the radial distortion parameters





Pincushion distortion

Camera calibration

- It consists in calculating internal and external parameters for a specific camera, including the distortion parameters
- The procedure is as follows:
  - Print a pattern and attach it to a planar surface (normally a checkerboard pattern is used)
  - Take a few images of the model plane under different orientations by moving either the plane or the camera
  - Detect the feature points in the images
  - Estimate the intrinsic and extrinsic parameters using the closed-form solution as described in the following
  - Estimate the coefficients of the radial distortion by solving the linear least-squares
  - Refine all parameters, including lens distortion parameters, by minimizing procedure

Closed form solution for parameter estimation

- Considering a feature point in the world reference frame and the mapped point in the camera reference frame we have:

Scaling factor

Camera reference frame

$$s\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K[R \mid t]\begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

world reference frame

- We assume the image is such that Z=0, therefore, setting $R = [r_1\ r_2\ r_3]$

$$s\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K[r_1\ r_2\ r_3 \mid t]\begin{pmatrix} X \\ Y \\ 0 \\ 1 \end{pmatrix} = \lambda K[r_1\ r_2 \mid t\ ]\begin{pmatrix} X \\ Y \\ 1 \end{pmatrix}$$

Closed form solution for parameter estimation

- We define $\lambda K[r_1\ r_2\ t] = H = [h_1\ h_2\ h_3]$

- Then, by knowing that r_1 and r_2 are orthogonal, we have:

$$\begin{cases} h_1^T K^{-T} K^{-1} h_2 = 0 \\ h_1^T K^{-T} K^{-1} h_1 = h_2^T K^{-T} K^{-1} h_2 \end{cases}$$

- The equations has 8 degrees of freedom (camera internal + external parameters)

- By combining the equations for two independent images, we can computer the camera internal parameters K

Closed form solution for parameter estimation

- Once K is known, we can compute the rotation matrix as follows:

$$
\begin{aligned}
r_1 &= \lambda K^{-1} h_1 \\
r_2 &= \lambda K^{-1} h_2 \\
r_3 &= r_1 r_2 \\
t &= \lambda K^{-1} h_3
\end{aligned}
$$

- Where $\lambda = \frac{1}{\|K^{-1}h_1\|} = \frac{1}{\|K^{-1}h_2\|}$

Estimate the coefficients of the radial distortion

- The radial distortion parameters (k_1, k_2)can be estimated by solving $2nm$ equations, where each ideal pixel position (u, v) is computed starting from the actual pixel position (x, y), while considering m points in n images

$$\begin{bmatrix} (u-u_0)(x^2+y^2) & (u-u_0)(x^2+y^2)^2 \\ (v-v_0)(x^2+y^2) & (v-v_0)(x^2+y^2)^2 \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} = \begin{bmatrix} \breve{u}-u \\ \breve{v}-v \end{bmatrix}$$

Refine all parameters

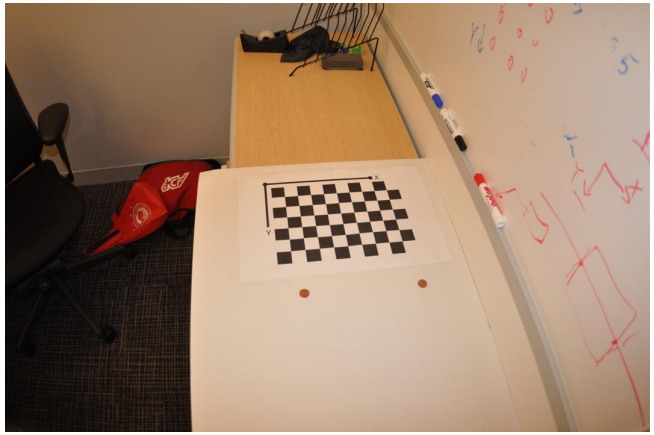
- Given n images of a model plane and m points on the model plane
- The maximum likelihood estimate of the parameters can be obtained by minimizing the following functional

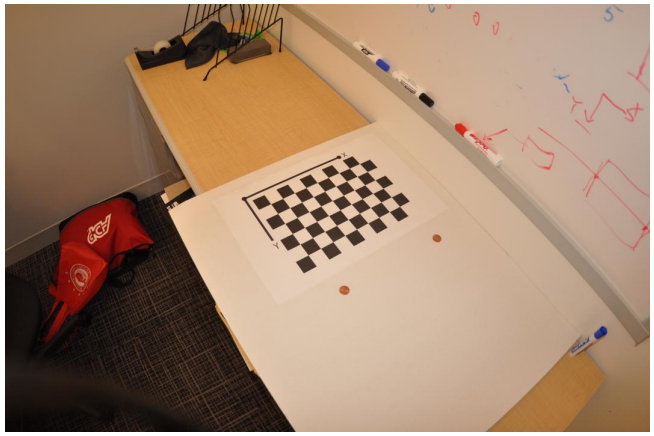
$$\sum_{i=1}^{n}\sum_{j=1}^{m}\|\mathbf{m}_{ij}-\widehat{\mathbf{m}}(\mathbf{A},\mathbf{R}_i,\mathbf{t}_i,\mathtt{M}_j)\|^2$$

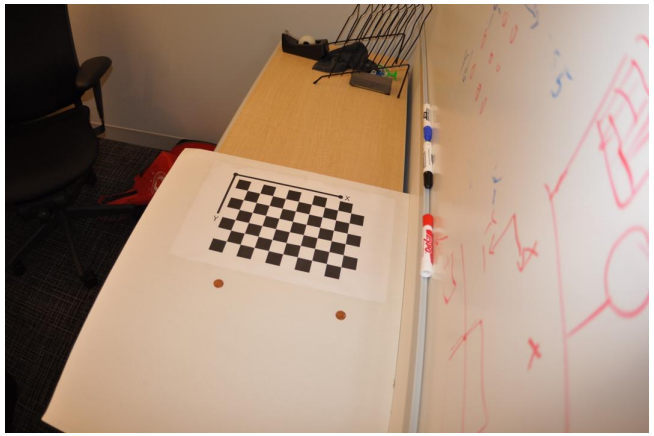
projection of point M_j in image i according to $\lambda K[r_1\ r_2 \mid t\ ]M$

Camera calibration in MATLAB

- Nine test images consisting of checkerboard pattern
  - 1072x712 pixels
  - 7x10 checkerboard
  - Each square is 29mm wide



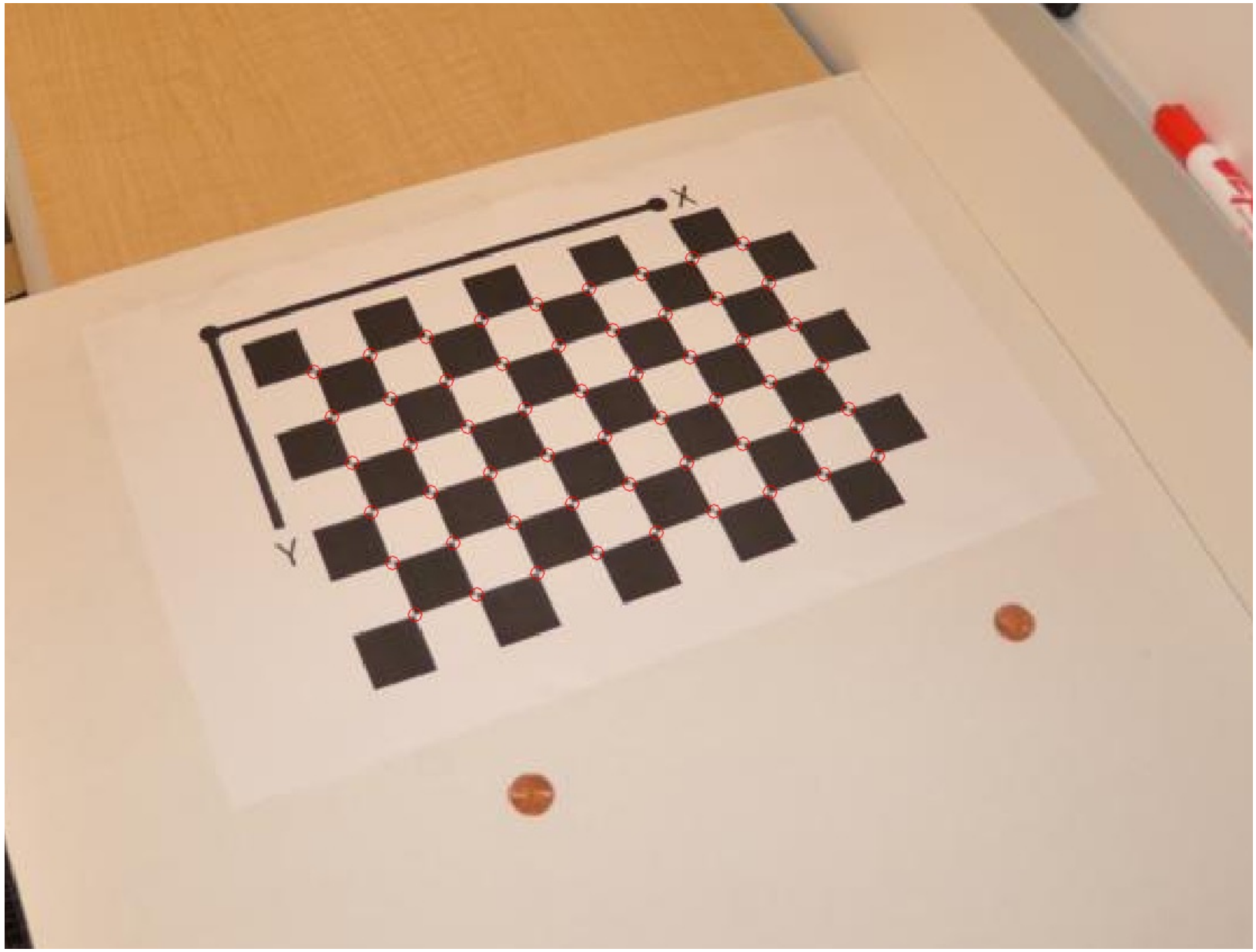




Camera calibration in MATLAB

- Load images and detect checkerboard points, coordinates are in the camera reference plane

```
images = imageSet(fullfile(toolboxdir('vision'),'visiondata','calibration','mono'));
imageFileNames = images.ImageLocation;
[imagePoints, boardSize] = detectCheckerboardPoints(imageFileNames);
```


X
Y

Camera calibration in MATLAB

- Generate checkboard points in the world reference plane

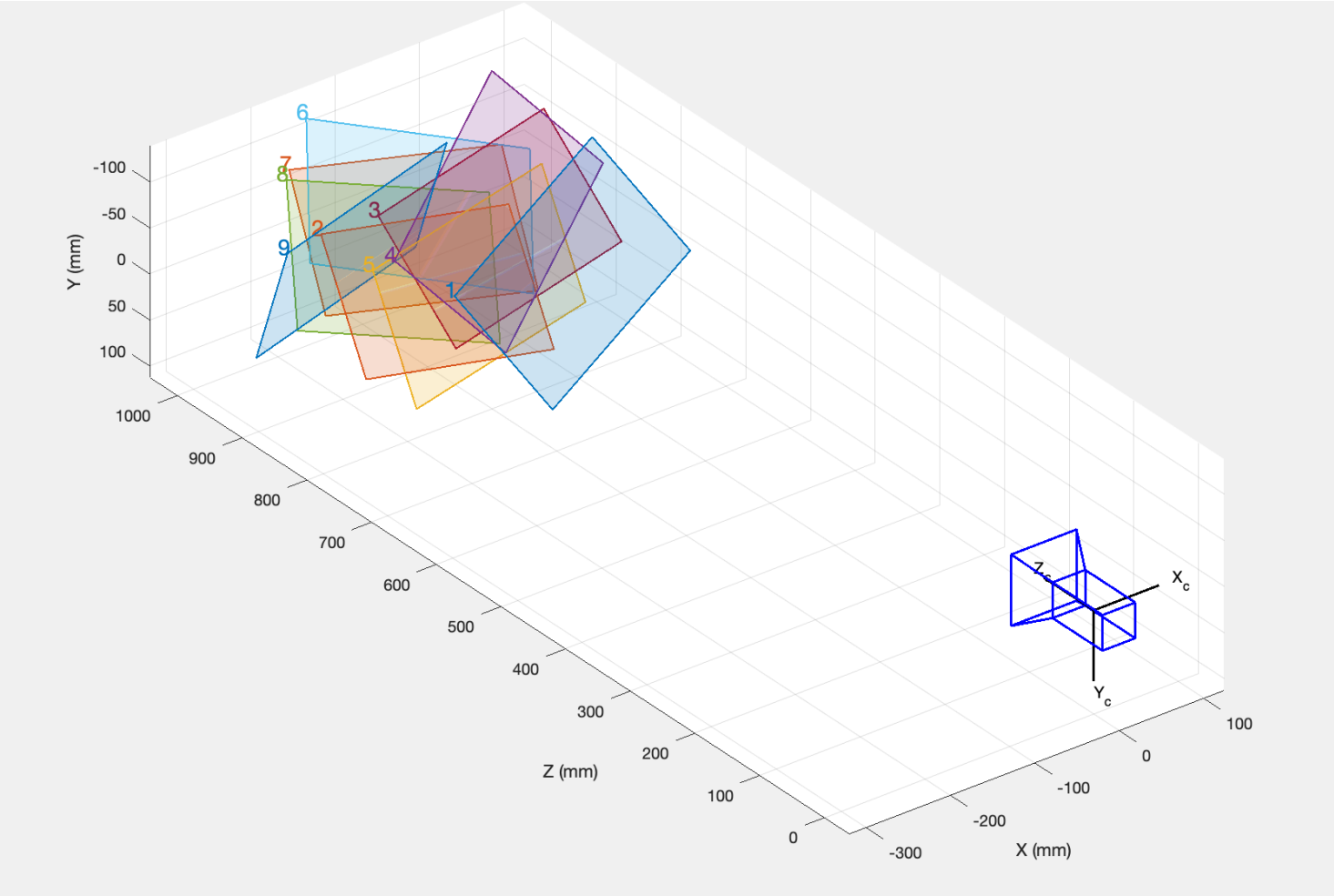
```
squareSizeInMM = 29;
worldPoints = generateCheckerboardPoints(boardSize,squareSizeInMM);
```

- Perform calibration

```
I = readimage(images,1);
imageSize = [size(I, 1),size(I, 2)];
params = estimateCameraParameters(imagePoints,worldPoints,'ImageSize',imageSize);
```

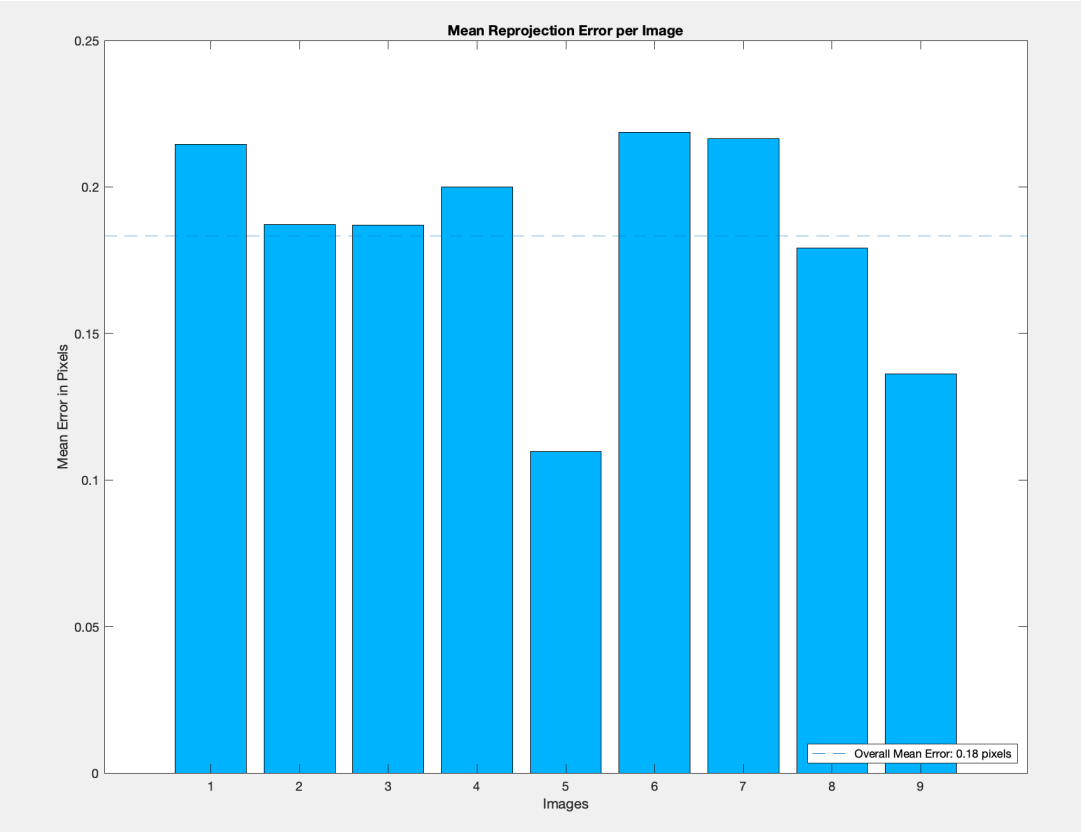
- This operation will return the internal and external camera parameters

Representation of external parameters



Parameter estimation error

- The parameter estimation is an optimization process; therefore, an estimation error is to be expected



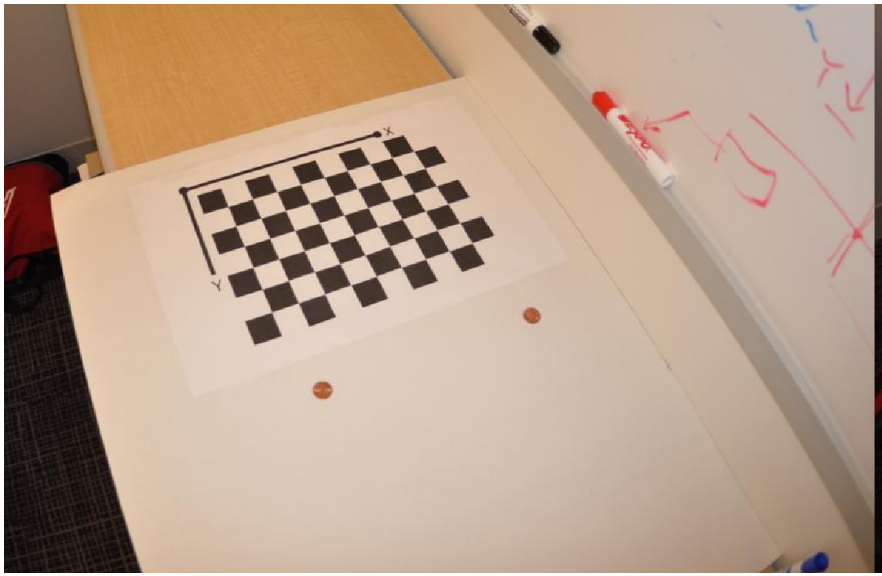
Outline

- Introduction
- The image pipeline
  - Camera calibration
  - Distortion correction
  - Perspective transformation
  - Lane detection

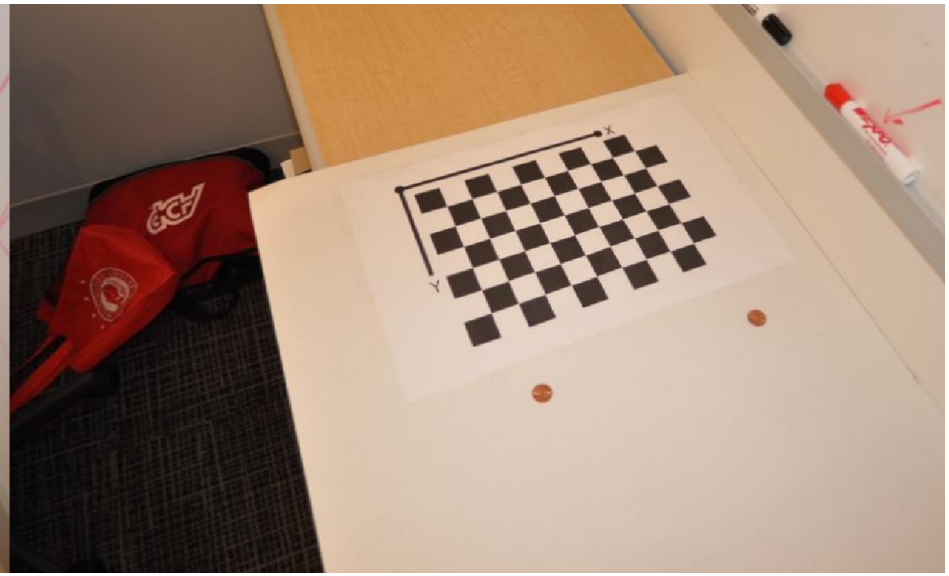
Distortion correction

- After camera calibration the distortions affecting the image can be removed by applying the radial distortion parameters

Original image



Undistorted image



Distortion correction in MATLAB

- After camera calibration

```
J1 = undistortImage(I,params);
```

Outline

- Introduction
- The image pipeline
  - Camera calibration
  - Distortion correction
  - Perspective transformation
  - Lane detection

Perspective transformation

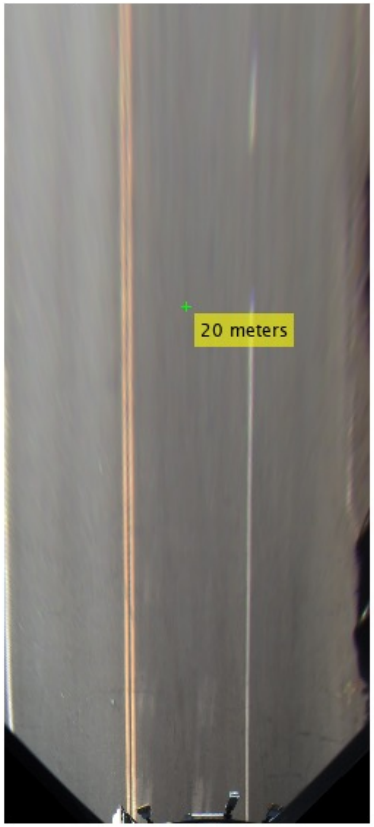
- The image obtained by the car camera contains a large non-road area, such as sky, trees on roadside, etc.
- Processing of the image will increase the computational complexity and reduce the real-time capability
- The invalid area will interfere the lane information and affect the detection accuracy
- Only the area shown in the red box is the region of interest (ROI) for lane detection

Perspective transformation

- Due to the imaging perspective effect, the traffic lanes are two non-parallel lines
- The inverse perspective transformation eliminates the perspective effect and produces a top view image that is consistent with the actual situation
- In the resulting top view image, the lane lines are vertical and parallel to each other, which facilitate the identification in subsequent algorithms

Original Image

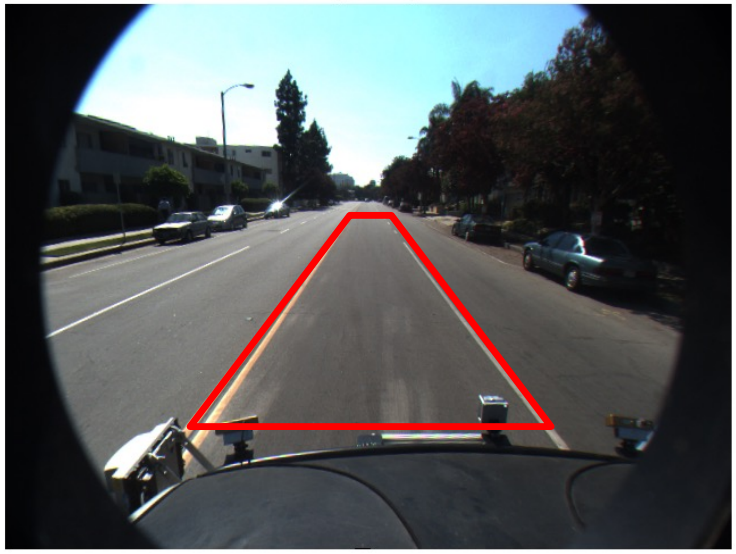
Bird's Eye View



Perspective transformation

- The ROI should be converted into an inverted trapezoidal shape after the transformation

- The inverse perspective transformation is the process that convert a rectangular into an inverted trapezoidal



Perspective transformation

- (u, v) denotes the ROI coordinate system
- (x, y) denotes the transformed top view coordinate system
- The mapping relationship can be expressed as

$$P = QM$$

- Where:
  - $P = [x' \;\; y' \;\; w']$
  - $x = \frac{x'}{w'}, y = \frac{y'}{w'}$
  - $Q = [u \;\; v \;\; 1]$
  - $M = \begin{bmatrix} a & d & g \\ b & e & h \\ c & f & 1 \end{bmatrix}$, the transformation matrix from the rectangular to the inverted trapezoidal

Perspective transformation in MATLAB

- Knowing the internal and external camera parameters, we have to:
  - Create a `monoCamera` sensor object
  - Define the ROI
  - Compute the inverse trapezoidal transformation using the `birdsEyeView()` operator
  - Finally, we can apply the transformation to the input image

```
focalLength = [309.4362 344.2161];
principalPoint = [318.9034 257.5352];
imageSize = [480 640];
camIntrinsics =
cameraIntrinsics(focalLength,principalPoint,imageSize);
height = 2.1798; pitch = 14;
sensor = monoCamera(camIntrinsics,height,'Pitch',pitch);

distAhead = 30;
spaceToOneSide = 6;
bottomOffset = 3;
outView = [bottomOffset,distAhead,-
spaceToOneSide,spaceToOneSide];

outImageSize = [NaN,250];
birdsEye = birdsEyeView(sensor,outView,outImageSize);

I = imread('road.png');
BEV = transformImage(birdsEye,I);
```

Outline

- Introduction
- The image pipeline
  - Camera calibration
  - Distortion correction
  - Perspective transformation
  - Lane detection

Lane detection

- It consists in the following steps:
  - Grayscale conversion and noise reduction for lane enhancement
  - Image binarization
  - Extraction of lane characteristics

Grayscale conversion and noise reduction

- Image is transformed to grayscale to reduce the amount of information to process
- Random noise disturb the image acquired by the camera
- It may influence the results of lane detection
- Gaussian blur is used to remove the noise
  - Low pass filter
  - A Gaussian function is used for calculating the transformation to apply to each pixel in the image

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Convolution

- The Gaussian blur filter is defined by a convolution matrix called kernel
- The filter is applied using matrix multiplication

Filtered image

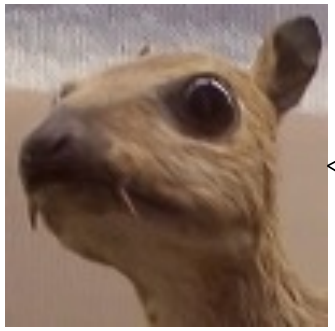
$$g(x,y) = w \cdot f(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) \cdot f(x-s, y-t)$$

Kernel

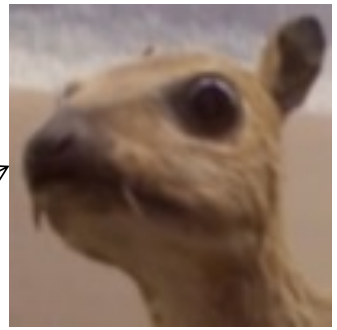
Source image

- w s and t are selected according to a specific goal
- For the Gaussian blur w is either 3x3 or 5x5

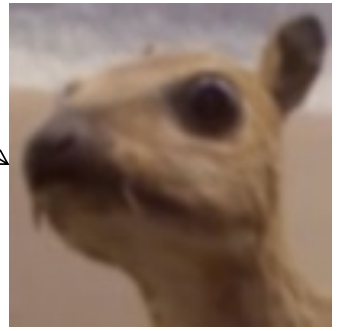
Gaussian blur



Original image



$$w = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



$$w = \frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

Lane enhancement

- For efficient implementation two separable kernels are used

$$g(x) = \frac{1}{\sigma_x^2} e^{\left(-\frac{x^2}{2\sigma_x^2}\left(1-\frac{x^2}{\sigma_x^2}\right)\right)}$$

$$g(y) = e^{\left(-\frac{1}{2\sigma_y^2}y^2\right)}$$

Image binarization

- To distinguish lane lines from background, image is binarized according to the grayscale of the image pixels

$$b(x,y) = \begin{cases} 255 \implies g(x,y) \geq Th \\ 0 \implies g(x,y) < Th \end{cases}$$

Image binarization

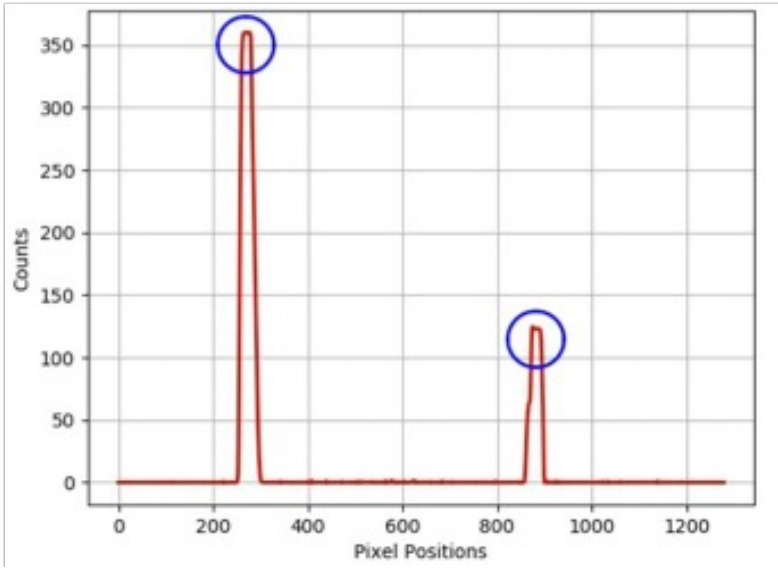
- The key to binarize an image is how to select the threshold Th
- Since illumination varies under deferent road scenes, it is impossible to distinguish the lane lines from background using a fixed threshold
- An iterative method is used to determine an optimal threshold to extract lane lines
- The optimal threshold is capable of adapting to varied illumination

Image binarization

- Let set $Th_0 = \frac{g_{max}+g_{min}}{2}$ where g_{max} and g_{min} are the maximum and minimum gray value within the original image respectively
- Then, the image is divided in two regions, A and B and the average gray value of the two regions is calculated as follows, starting with $i = 0$

$$g_A = \frac{\sum_{g(x,y)\geq Th_i} g(x,y)}{\sum_{g(x,y)\geq Th_i} 1} \qquad g_B = \frac{\sum_{g(x,y)<Th_i} g(x,y)}{\sum_{g(x,y)<Th_i} 1}$$

- Then, the threshold is updated as $Th_{i+1} = \frac{g_A+g_B}{2}$
- The process is repeated until $Th_{i+1} = Th_i$

Extraction of lane characteristics

- In the binary image, the pixels corresponding to the line are 255, while the others are 0
- The pixels in each column of the image are summed and a histogram is built
- The position of the two peaks represents the position of the lane delimiters